

Tổng quan máy tính	2
Số nguyên - Số chấm động	3
Số lượng dấu, bù 1, bù 2, K access	3
Số chấm động	6
Ví dụ tính toán	8
Bộ xử lý + Hợp ngữ	9
x86 (so sánh với MIPS)	16
Mạch số	16
Hệ thống lưu trữ (Cache)	24
Hệ thống nhập xuất	30
Bảng lũy thừa của 2	31

Tổng quan máy tính

Các thế hệ của máy tính

- Thế hệ 1 – Bóng đèn chân không (Vacuum Tube) (1946-1957)

Gồm: Vacuum Tube, IBM 700, ENIAC (Electronic Numerical Integrator And Computer)

- Thế hệ 2 - Transistor (1958 - 1964)

Gồm: IBM 7094

- Thế hệ 3 – Integrated Circuit (IC) (1965-1971)

Gồm: IBM 360

- Thế hệ 4 – Micro (1972-1977)

Gồm: Intel 4004 with 2300 transistors inside, XT computer with Intel 8086 chip

- **Quy luật Moore - Gordon Moore:** “số lượng transistor trên mỗi đơn vị inch vuông sẽ tăng lên gấp đôi sau mỗi 24 tháng”

- Kiến trúc von Neumann - còn được gọi là mô hình von Neumann hoặc kiến trúc Princeton - là kiến trúc máy tính dựa trên mô tả năm 1945 của nhà toán học và vật lý John von Neumann.

- Đặc điểm về kiến trúc của Von Neumann:

- + Dữ liệu và chương trình chứa trong bộ nhớ đọc ghi.
- + Bộ nhớ được đánh địa chỉ cho các ngăn nhớ không phụ thuộc và nội dung của chúng.
- + Các lệnh được máy tính thực hiện một cách tuần tự.

- Von Neumann chia hoạt của máy tính thành 5 thành phần chính là:

1. CPU (Bộ xử lý trung tâm)
2. Bộ nhớ làm việc (RAM)
3. Bộ nhớ vĩnh cửu (ROM)
4. Thiết bị vào (Input)
5. Thiết bị ra (Output)

- **Thùng máy:** hay đc sử dụng: ATX, ít sd: AT

- **Bo mạch chủ:** Mainboard/Motherboard - gắn CPU, card đồ họa, ...

- **FSB - Front side bus:** truyền giữa CPU - Chipset cầu Bắc



Chipset

- Là “cầu nối” giữa CPU và các thiết bị khác
- Thường gồm 2 thành phần: **chipset cầu bắc** (North Bridge Chipset) và **chipset cầu nam** (South Bridge Chipset)
- Chipset cầu bắc và chipset cầu nam quyết định sự tương thích của bo mạch với CPU
- Chipset cầu bắc đảm nhiệm việc liên lạc giữa CPU, RAM, AGP (hoặc PCI Express) và chipset cầu nam
- Chipset cầu nam đảm nhiệm những việc có tốc độ chậm như liên kết với card âm thanh, card mạng, ổ cứng, USB...

Số nguyên - Số chấm động

Công thức đổi bit: (ưu tiên chọn các toán tử chính (AND OR NOT XOR))

- Get the value of bit i: $(x \text{ SHR } i) \text{ AND } 1$
- Set the value 1's of bit i: $(1 \text{ SHL } i) \text{ OR } x$ hoặc là $(1 \text{ SHR } (N - 1 - i)) \text{ OR } x$
- Set the value 0's of bit i : $\text{NOT}(1 \text{ SHL } i) \text{ AND } x$
- Reverse bit i: $(1 \text{ SHL } i) \text{ XOR } x$

Số lượng dấu, bù 1, bù 2, K access

1 số lưu ý

- Phép AND: chuyển dãy bit thành dãy bit nhỏ hơn
- Phép OR: chuyển dãy bit thành dãy bit lớn hơn
- Phép XOR: 2 bit xor = 0 khi nó giống nhau
- 2 dãy bit giống nhau XOR với nhau = 0
- n bit xor với nhau kết quả = 1 nếu k bit 1 lẻ
- AND với 0 ra 0, AND với 1 ra chính nó
- OR với 1 ra 1, OR với 0 ra chính nó
- XOR với 0 = chính nó, XOR với 1 = NOT nó

Số lượng dấu (8 bit)

100 : 0110 0100

-100: 1110 0100

Phạm vi: $-(2^{n-1} - 1) \rightarrow 2^{n-1} - 1$

Số bù 1 (8 bit)

100 : 0110 0100

-100: 1001 1011

Số bù 2 (8 bit)

100 : 0110 0100

-100: 1001 1100

Số bias (8 bit)

100 : 1110 0011

-100: 0001 1011

Nhận biết tràn số: nhớ vào nhưng không nhớ ra, ngược lại

$(-) + (-) = (+)$ hoặc $(+) + (+) = (-)$

Nhớ vào: Bit kế bên bit phải cùng có nhớ (vd: 0100 + 0100)

Nhớ ra: Bit phải cùng cộng có nhớ (vd: 1000 + 1000)

Phạm vi biểu diễn số bù 1: -126 $\rightarrow$ 126 $-(2^{n-1} - 1) \rightarrow 2^{n-1} - 1$ //dấu ngược đúng chỗ nha!!!!

Khi cộng mà dư ra 1 ở đầu, thì phải đưa 1 về cuối để thực hiện tiếp: (không phải tràn số)

Vd : 1101 + 1011 = 1 1000 = 1000 + 1 = 1001

$(-2) + (-4) \qquad \qquad \qquad (-6)$

Phạm vi biểu diễn số bù 2: $-2^{n-1} \rightarrow 2^{n-1} - 1$

Phép trừ số bù 2: chuyển thành phép cộng cho số âm

Bias: $-(2^{(n-1)}-1) \rightarrow 2^{(n-1)}$ với $K=2^{(n-1)}-1$

Shift (logic) left kết quả = nhân 2^k

Shift (logic) right kết quả = chia 2^k

Shift logic: Thêm số 0 vào chỗ trống

Vd : 1001 -> dịch trái : 0010

-> dịch phải: 0100

Shift arithmetic: dịch phải thêm bit dấu

Vd: 1001 -> dịch trái: 0010

-> dịch phải: 1100 (bit dấu là 1)

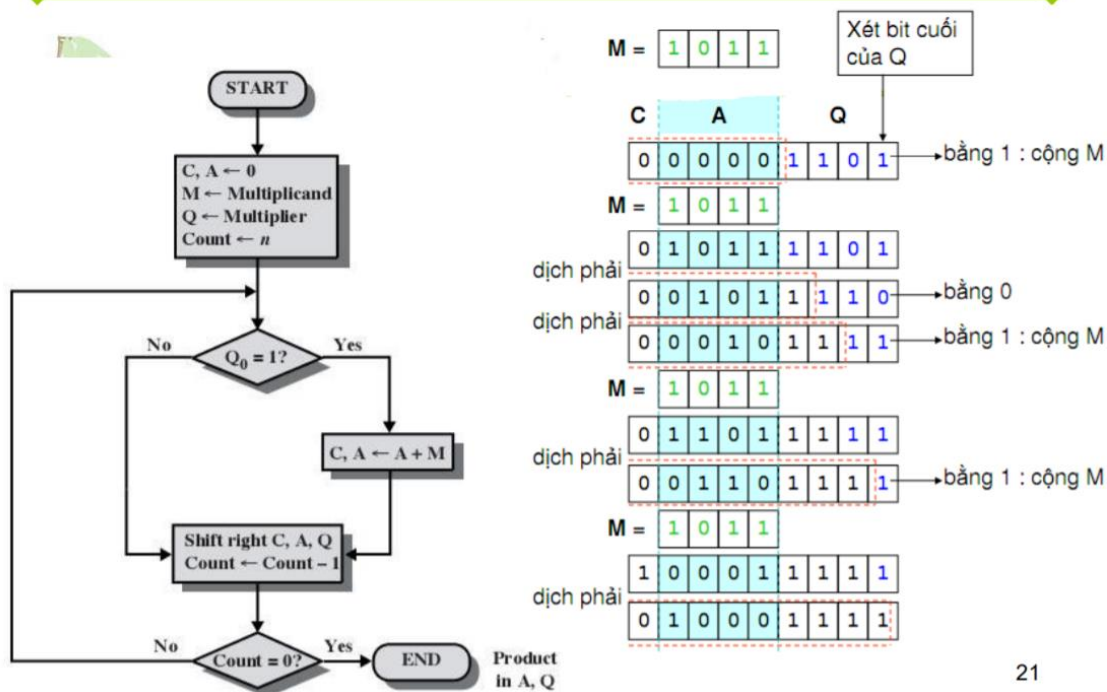
Vd: 0110 -> dịch trái: 1100

-> dịch phải: 0011 (bit dấu là 0)



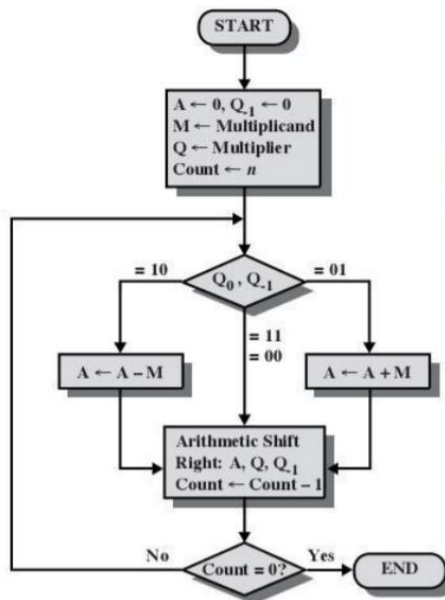
Khoa
CÔNG NGHỆ THÔNG TIN
ĐH Khoa học Tự nhiên TP HCM

Thuật toán nhân không dấu



21

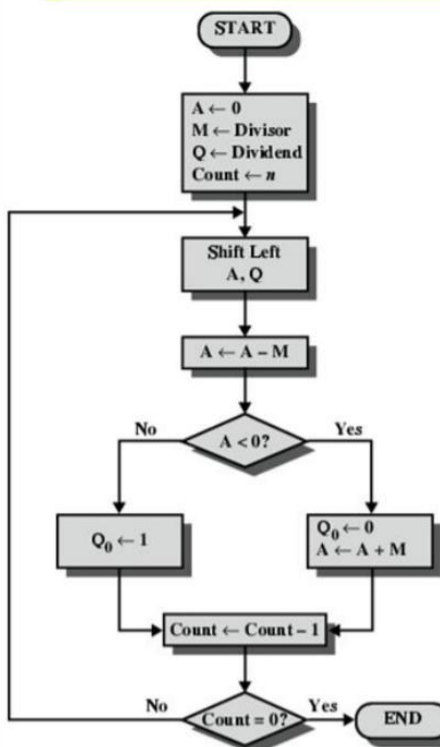
Thuật toán Booth – Ví dụ



	A	Q	Q ₋₁	M
Khởi đầu	0000	1101	0	0111
Bước 0: A=A-M	1001	1101	0	0111
shift	1100	1110	1	0111
Bước 1: A=A+M	0011	1110	1	0111
shift	0001	1111	0	0111
Bước 2: A=A-M	1010	1111	0	0111
shift	1101	0111	1	0111
Bước 3: shift	1110	1011	1	0111

Kết quả 11101011 = -21

Thuật toán chia không dấu

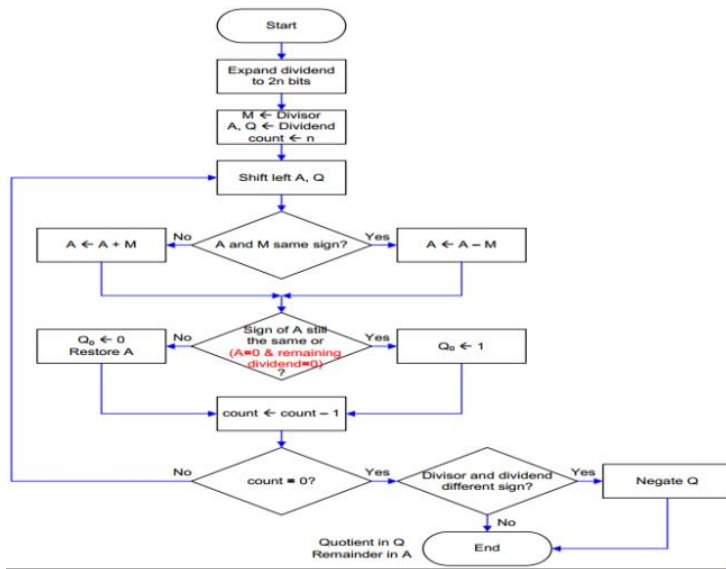


A	Q	M = 0011
0000	0111	Initial value
0000	1110	Shift
1101		Subtract
0000	1110	Restore
0001	1100	Shift
1110		Subtract
0001	1100	Restore
0011	1000	Shift
0000		Subtract
0000	1001	Set Q ₀ = 1
0001	0010	Shift
1110		Subtract
0001	0010	Restore

(7)/(3)

Quotient in Q
Remainder in A

Divide Operation – Two's Complement Numbers



A	Q	M = 1101
0000	0111	Initial value
0000	1110	Shift
1101	1110	Add
0000	1110	Restore
0001	1100	Shift
1110	1100	Add
0001	1100	Restore
0011	1000	Shift
0000	1000	Add
0000	1001	Set $Q_0 = 1$
0001	0010	Shift
1110	0010	Add
0001	0010	Restore

(7)/(-3)

53

Số chấm động



Khoa
CÔNG NGHỆ THÔNG TIN
ĐHQG TP HCM

Bảng tóm tắt số chấm động

	Single Precision (32 bit)				Double Precision (64 bit)			
	Dấu	Mũ	Trị	Giá trị	Dấu	Mũ	Trị	Giá trị
+0	0	0	0	0	0	0	0	0
-0	1	0	0	-0	1	0	0	-0
$+\infty$	0	255 (toàn bit 1)	0	∞	0	2047 (toàn bit 1)	0	∞
$-\infty$	1	255 (toàn bit 1)	0	$-\infty$	1	2047 (toàn bit 1)	0	$-\infty$
Quiet NaN	0/ 1	255 (toàn bit 1)	$\neq 0$	NaN	0/ 1	2047 (toàn bit 1)	$\neq 0$	NaN
Signaling NaN	0/ 1	255 (toàn bit 1)	$\neq 0$	NaN	0/ 1	2047 (toàn bit 1)	$\neq 0$	NaN
Số dương (dạng chuẩn)	0	$0 < e < 255$	f	$2^{e-127} (1.f)$	0	$0 < e < 2047$	f	$2^{e-1023} (1.f)$
Số âm (dạng chuẩn)	1	$0 < e < 255$	f	$-2^{e-127} (1.f)$	1	$0 < e < 2047$	f	$-2^{e-1023} (1.f)$
Số dương (dạng không chuẩn)	0	0	$f \neq 0$	$2^{-126} (0.f)$	0	0	$f \neq 0$	$2^{-1022} (0.f)$
Số âm (dạng không chuẩn)	1	0	$f \neq 0$	$-2^{-126} (0.f)$	1	0	$f \neq 0$	$-2^{-1022} (0.f)$

Description	exp	frac	Single precision		Double precision	
			Value	Decimal	Value	Decimal
Zero	00...00	0...00	0	0.0	0	0.0
Smallest denorm.	00...00	0...01	$2^{-23} \times 2^{-126}$	1.4×10^{-45}	$2^{-52} \times 2^{-1022}$	4.9×10^{-324}
Largest denorm.	00...00	1...11	$(1 - \epsilon) \times 2^{-126}$	1.2×10^{-38}	$(1 - \epsilon) \times 2^{-1022}$	2.2×10^{-308}
Smallest norm.	00...01	0...00	1×2^{-126}	1.2×10^{-38}	1×2^{-1022}	2.2×10^{-308}
One	01...11	0...00	1×2^0	1.0	1×2^0	1.0
Largest norm.	11...10	1...11	$(2 - \epsilon) \times 2^{127}$	3.4×10^{38}	$(2 - \epsilon) \times 2^{1023}$	1.8×10^{308}

Chap2, Prentice-Hall Computer Systems: A Programmer's Perspective, 2nd, 2011, Figure 2.35



Chuyển từ biểu diễn thập phân sang nhị phân

$$-2.340625 \times 10^1$$

1. Không chuẩn hóa: -23.40625

2. Chuyển phần nguyên:

$$23 = 16 + 4 + 2 + 1 = 10111$$

3. Chuyển phần thập phân:

$$.40625 = .25 + .125 + .03125 = .01101$$

4. Kết hợp và chuẩn hóa:

$$10111.01101 = 1.011101101 \times 2^4$$

5. Chuyển phần mũ: $127 + 4 = 10000011$

1	1000 0011	011 1011 0100 0000 0000 0000
---	-----------	------------------------------



Các số đặc biệt

- Phần mũ = 0, phần trị = 0
 - Số zero
- Phần mũ = 0, phần trị $\neq 0$
 - Số dạng không chuẩn (denormalized)
- Phần mũ toàn bit 1, phần trị = 0
 - Số vô cùng (infinity)
- Phần mũ toàn bit 1, phần trị $\neq 0$
 - Số báo lỗi (NaN - Not a Number)
 - Signaling NaN
 - Quiet NaN

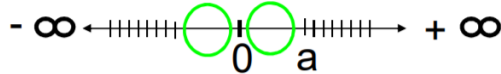


Số dạng không chuẩn

- Số dương nhỏ nhất có thể biểu diễn

$$a = 1.0 \dots_2 \times 2^{-126} = 2^{-126}$$

Gaps!



Lý do: ngầm định 1 + phần trị

- Giải pháp:
 - Quy ước nếu số mũ = 0 (phần trị $\neq 0$), không ngầm định bắt đầu là 1 $\rightarrow$ Số dạng không chuẩn (denormalized)
 - Số dương nhỏ nhất có thể biểu diễn
 - $a = 0.00 \dots 1_2 \times 2^{-126} = 2^{-23} \times 2^{-126} = 2^{-149}$



- Ví dụ tính toán

Chuyển thập phân sang nhị phân

- 20.5

Chuyển phần nguyên: 10100

Chuyển phần thập phân: $0.5 \times 2 = 1$

$10100.1 \rightarrow 1.01001 \times 2^4$

Chuyển phần mũ: $4 + 127 = 1000\ 0011$

0 | 1000 0011 | 010 0100 0000 0000 0000

- 0.1

Chuyển phần thập phân: $0.1 \rightarrow 0.2 \rightarrow 0.4 \rightarrow 0.8 \rightarrow 1.6 \rightarrow 1.2 \rightarrow 0.4 \rightarrow 0.8 \rightarrow 1.6 \rightarrow \dots$

$\rightarrow 0.000110011001\dots$

$\rightarrow 1.10011001 \times 2^{-4}$

Chuyển phần mũ: $-4 + 127 = 01111011$

0 | 01111011 | 10011001100110011001100

- 7×2^{-130}

Chuyển phần nguyên: 7: 111

Kết hợp chuẩn hóa: $111 \times 2^{-130} = 0.0111 \times 2^{-126}$

Phần mũ: **đang không chuẩn hóa**

0 | 0000 0000 | 011100000000000000000000

Chuyển nhị phân sang thập phân

0101 1111 1011 1110 0100 0000 0000 0000

$\rightarrow$ Chuyển phần mũ: $1011\ 1111 = 191 - 127 = 64$

$\rightarrow$ Chuyển phần nguyên: 011 1110 0100 0000 0000 0000

$1 + 2^{-2} + 2^{-3} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-9} = 1.486328125$

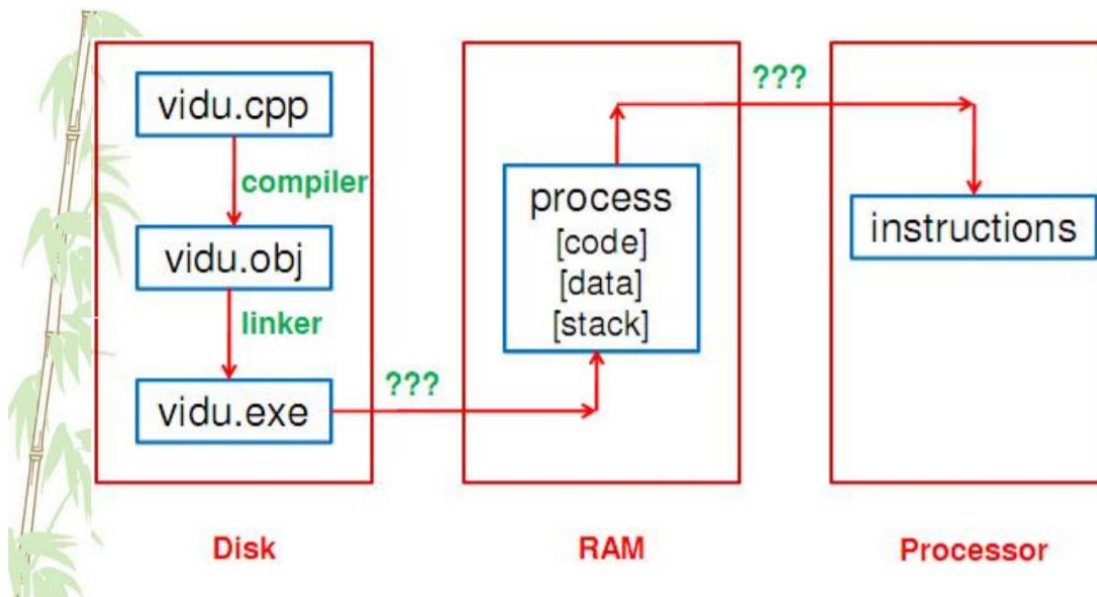
Kết quả = $1.486328125 \times 2^{64}$

Bộ xử lý + Hợp ngữ

Cấu trúc máy tính:

- Bộ xử lý (Central processing unit)
- Bộ nhớ chính (Main memory)
- Hệ thống kết nối (Bus)
- Thiết bị nhập xuất (Input - Output)

Thực thi chương trình



Bộ xử lý:

- Thanh ghi (Register): Lưu trữ tạm dữ liệu để ALU, CU xử lý và điều khiển.
- Đơn vị xử lý (Arithmetic & Logic Unit): Xử lý, tính toán trên dữ liệu của thanh ghi.
- Internal Bus: Kết nối giữa các thành phần CU, ALU và Register trong CPU.
- Đơn vị điều khiển (Control Unit): Điều khiển xử lý của ALU và dữ liệu trên thanh ghi.

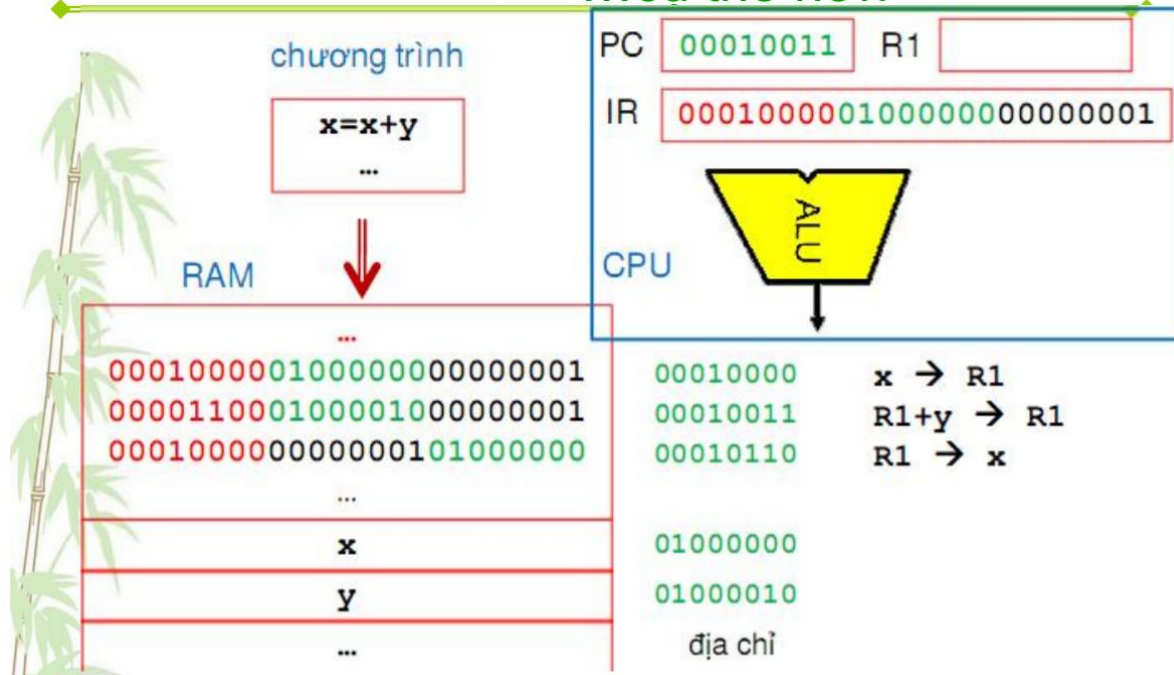
Lệnh máy (machine instruction/ code) là dãy bit chứa yêu cầu mà bộ xử lý phải thực hiện. Cấu trúc thường gồm:

- Mã thao tác (opcode): cho biết lệnh thực hiện thao tác gì (+, -, and, or, ...)
- Các toán hạng (operand): cho biết các đối tượng bị tác động bởi thao tác trong mã thao tác (thanh ghi, vùng nhớ, hằng số, ...)

Thanh ghi PC (Program Counter) – Lưu địa chỉ của lệnh sẽ được nạp

Thanh ghi IR (Instruction Register) – Lưu lệnh sẽ được xử lý.

Thực thi chương trình...
...cụ thể hơn



MIPS

4 nguyên tắc thiết kế bộ lệnh MIPS

- Cấu trúc lệnh đơn giản và có quy tắc (Simplicity favors regularity)
- Lệnh và bộ lệnh càng nhỏ gọn càng xử lý nhanh (Smaller is faster)
- Tăng tốc độ xử lý cho những trường hợp thường xuyên xảy ra (Make the common case fast)
- Thiết kế tốt đòi hỏi sự thỏa hiệp tốt (Good design demands good compromises)

- MIPS chỉ hỗ trợ 32 thanh ghi
- 1 thanh ghi có kích thước 32 bit (4 byte)
- Trong MIPS, nhóm 32 bit được gọi là **1 word**

Tại sao 2 trường opcode và funct không nằm liền nhau?

- Mục đích là người ta sẽ bỏ trường function để tạo thêm 1 cấu trúc mới nên không để nó liền kề nhau.

Tại sao không dùng rt làm số bit dịch?

- Tại vì mỗi trường thực hiện đúng chức năng của nó Vì rt là trường cho địa chỉ thanh ghi nếu nó làm số bit dịch thì nó mang 2 vai trò trong các trường hợp khác nhau.

Tại sao không xây dựng các lệnh MIPS có nhiều toán hạng nguồn hơn?

- * Do cấu trúc MIPS cố định là 32 bit, và chủ trương của MIPS là làm các lệnh đơn giản để tăng tốc độ xử lý của chương trình, việc xây dựng lệnh có nhiều mã toán hạng nguồn sẽ tăng độ phức tạp của đoạn lệnh lên (câu này bao quát hơn :jzb:)
- Vì lệnh trong MIPS cần sự đơn giản nên 2 toán hạng nguồn là đủ (câu này thầy trả lời)

Tại sao không có lệnh gán trực tiếp giá trị của 1 thanh ghi vào 1 thanh ghi?

- Vì lệnh trong MIPS cần sự đơn giản nên không nhất thiết phải thêm lệnh, đồng thời nếu có lệnh gán thì nó chỉ sử dụng 2 thanh ghi vì phạm nguyên tắc ban đầu của MIPS (quy tắc 3 thanh ghi)

Tại sao không có lệnh not mà lại sử dụng lệnh nor thay cho lệnh not?

- $\text{Not}(A) = A \text{ nor } 0$.

Tại sao không có các lệnh tính toán luận lý còn lại như: xor, nand, ...?

- Tại vì các lệnh đó có thể rã ra thành các lệnh nhỏ như and, or, not giúp cho cấu trúc lệnh đơn giản, nhỏ gọn.

Trong MIPS, chỉ có các lệnh nạp, lưu mới sử dụng toán hạng vùng nhớ – Tại sao không sử dụng toán hạng vùng nhớ trong các lệnh khác như số học, luận lý, ...?

- Vì để cho mạch xử lý đơn giản hơn, dễ nâng cao tốc độ bằng cách sử dụng các kỹ thuật song song hóa

Tại sao lại cần các lệnh thao tác với hằng số trong khi các lệnh này đều có thể được thực hiện bằng cách kết hợp các lệnh nạp, lưu với các thao tác trên thanh ghi?

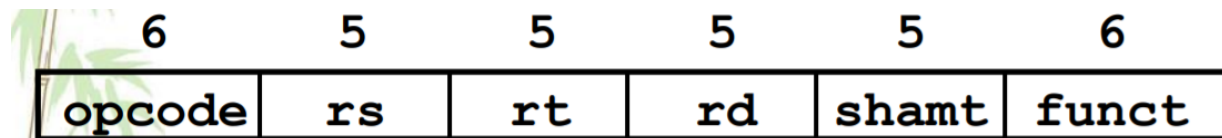
- Là để chương trình chạy nhanh hơn tại vì hằng số sẽ ko cần phải nạp vào thanh regis khác

Tại sao không có lệnh trừ hằng số, chẳng hạn subi?

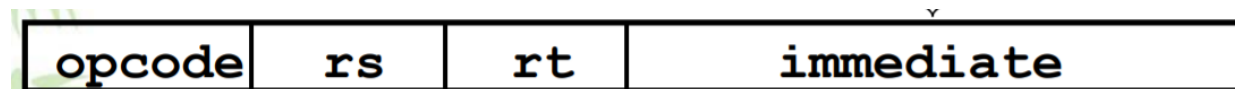
- Do có thể dùng addi với số âm nên ko có cần subi nữa

Chú ý các format: lw, sw, r-format và beq

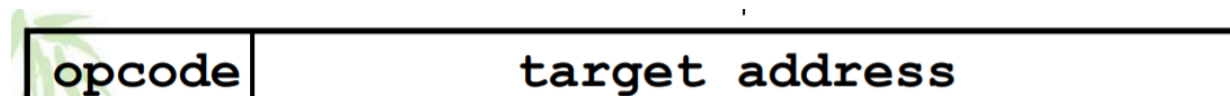
R - format



I - format



J - format

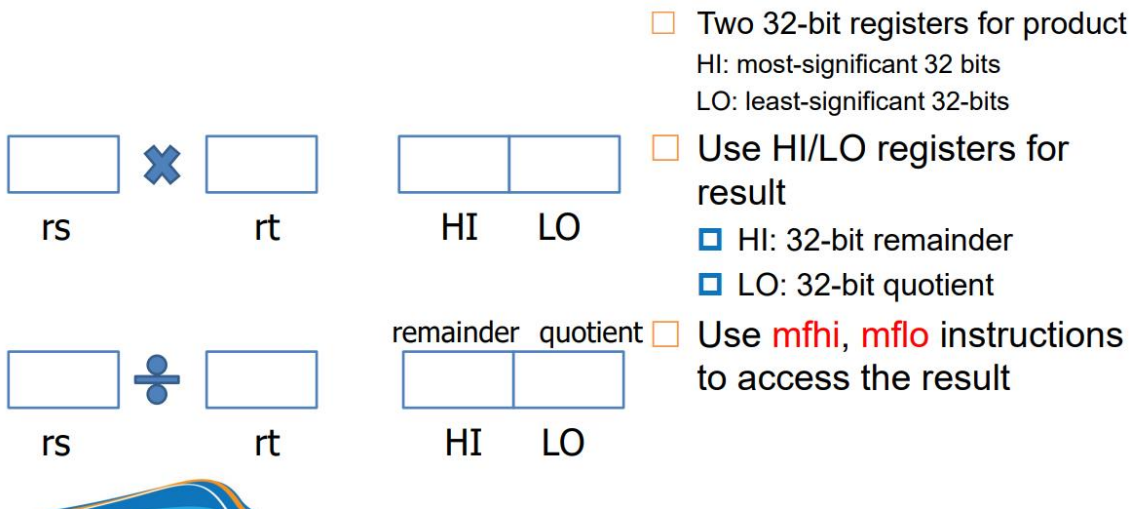


Dịch có 2 loại dịch:

- + Dịch logic sll - srl
- + Dịch số học chỉ có dịch phải sra (dịch trái ko có ý nghĩa vì luôn luôn làm tràn số nếu nó là số âm)
- Khi dịch ko dùng rs chỉ dùng rt, rd và shamt
- Lệnh dịch với lệnh cộng: opcode luôn = 0, function khác nhau
- opcode phân biệt lệnh có cấu trúc R-format với lệnh khác (lệnh 6 trường - cấu trúc R-format - opcode = 0)

Quy tắc Big Endian và Little Endian (vd số 0x12345678)

12 34 56 78
 3 2 1 0 (big)
 0 1 2 3 (little)



- Through the system call (**syscall**) instruction to request a service from a small set of the operating system services
- System call code** → **\$v0**
- Arguments** → **\$a0 - \$a3** (\$f12 for floating-point)
- Return value** → **\$v0** (\$f0 for floating-point)

Service	System call code	Arguments	Result
print_int	1	\$a0 = integer	
print_float	2	\$f12 = float	
print_double	3	\$f12 = double	
print_string	4	\$a0 = string	
read_int	5		integer (in \$v0)
read_float	6		float (in \$f0)
read_double	7		double (in \$f0)
read_string	8	\$a0 = buffer, \$a1 = length	
sbrk	9	\$a0 = amount	address (in \$v0)
exit	10		
print_char	11	\$a0 = char	
read_char	12		char (in \$v0)
open	13	\$a0 = filename (string), \$a1 = flags, \$a2 = mode	file descriptor (in \$a0)
read	14	\$a0 = file descriptor, \$a1 = buffer, \$a2 = length	num chars read (in \$a0)
write	15	\$a0 = file descriptor, \$a1 = buffer, \$a2 = length	num chars written (in \$a0)
close	16	\$a0 = file descriptor	
exit2	17	\$a0 = result	

50

Name	Register Number	Usage
\$zero	0	The constant value 0
\$v0	2	Result from callee
\$v1	3	Return to caller
\$a0	4	Argument to callee
\$a1-\$a3	5-7	From caller: caller saves
\$t0-\$t7	8-15	Temporaries: callee can clobber
\$s0-\$s7	16-23	Saved: callee can clobber
\$t8-\$t9	24-25	More temporaries (conditions)
\$gp	28	Global pointer
\$sp	29	Stack pointer
\$fp	30	Frame pointer
\$ra	31	Return address (caller saves)
\$at	1	Reserved for assembler
\$k0-\$k1	26-27	Reserved for the OS kernel

- Để thay đổi địa chỉ hoặc thông tin liên quan tới địa chỉ thì cấu trúc lệnh này gồm 2 giá trị so sánh với nhau

-> Cấu trúc lệnh mới I-format gom 3 trường rd, shamt và function thành 1 trường mới 16 bit là immediate, lúc này còn 1 trường đó là lệnh gì đó là trường opcode, xác định trường đó là trường gì thì dựa vào opcode, nếu = 0 thì đó là R-format chia thành 6 trường rồi dựa vào function. Còn nếu khác 0 thì dựa vào opcode để xác định lệnh **immediate** chứa khoảng cách so với địa chỉ nằm trong **thanh ghi PC** (Program Counter), thanh ghi chứa địa chỉ lệnh đang được thực hiện

- Lúc này trường **immediate** được xem như 1 số có dấu cộng với địa chỉ trong **thanh ghi PC** tạo thành địa chỉ cần nhảy tới
- Như vậy, có thể nhảy tới, lui 1 khoảng 215 (byte).
- Mỗi lệnh là 4 byte
- Như vậy, các lệnh rẽ nhánh có thể nhảy tới các địa chỉ có khoảng cách $\pm 2^{15}$ từ nhớ từ PC, có thể trong 1 cái if chứa khoảng 3200 lệnh máy

- Nếu không thực hiện rẽ nhánh:

$PC = PC + 4$

$PC + 4$ = địa chỉ của lệnh kế tiếp trong bộ nhớ

- Nếu thực hiện rẽ nhánh:

$PC = (PC + 4) + (\text{immediate} * 4)$

- Nắm được lệnh beq sử dụng ngay tại thời điểm đó đang làm gì và trường immediate trong lệnh beq đó là bao nhiêu vì đây là đặc điểm đặc trưng của MIPS mà x86 không hề có tại vì x86 các lệnh không có kích thước bằng nhau dẫn đến nó không thể nào chứa số lệnh nó nhảy qua được, chỉ có MIPS mỗi lệnh đều có 4 bytes thì cái trường trong lệnh nhảy mới chứa số lệnh mà nó nhảy qua. Chỉ có các cấu trúc thuộc **cấu trúc RISC** đơn giản và có quy tắc thì nó mới xây dựng được theo kiểu cấu trúc lệnh nhảy này. Lệnh nhảy trong x86 sẽ có một cách khác để tính. Đây là đặc điểm nổi bật của cấu trúc MIPS

- Với cấu trúc J-Format, có thể nhảy trong khoảng 2^{26} từ nhớ ($\sim 2^{28}$ byte)
- jal lưu địa chỉ của lệnh tiếp theo vào thanh ghi \$ra để sau khi thực hiện lệnh hàm ngoài main thì có thể trả về đúng vị trí

Tại sao lại thêm lệnh jal?

- Không cần phải xác định tường minh địa chỉ quay về trong \$ra
- \$sp con trỏ ngăn xếp trỏ vào đỉnh ngăn xếp, hoạt động ngược lại so với các vùng khác

<pre>main() { int a, b, c; c = sumSquare(a,b); } int sumSquare(int x, int y) { return mult(x,x) + y; } int mult(int x, int y) { ... } ----- main: add \$a0, \$s0, \$0 add \$a1, \$s1, \$0 jal sumSquare add \$s2, \$v0, \$0 ...</pre>	<pre>sumSquare: addi \$sp, \$sp, -8 sw \$ra, 4(\$sp) sw \$a1, 0(\$sp) add \$a0, \$a0, \$0 add \$a1, \$a0, \$0 jal mult lw \$a1, 0(\$sp) add \$v0, \$v0, \$a1 lw \$ra, 4(\$sp) addi \$sp, \$sp, 8 jr \$ra mult: add \$v0, \$0, \$0 Loop: slt \$t0, \$0, \$a1</pre>
--	--

```
beq $t0, $0, end
add $v0, $v0, $a0
addi $a1, $a1, -1
```

```
j Loop
end:
jr $ra
```

x86 (so sánh với MIPS)

MIPS	X86
“Kiến trúc 3 toán hạng” add \$s0,\$s1,\$s2 # s0=s1+s2 -> tốc độ nhanh hơn	“Kiến trúc 2 toán hạng” add EBX,EAX ; EBX=EBX+EAX ->Bộ nhớ ít hơn
“Kiến trúc nạp-lưu” lw \$t0, 12(\$gp) add \$s0, \$s0,\$t0 -s0=s0+Mem[12+gp]	“Kiến trúc thanh ghi - bộ nhớ” ADD EAX,[ESI + 12] -EAX=EAX+Mem[12+ESI]
“Các lệnh có chiều dài cố định”- 4byte Mạch xử lý đơn giản hơn Tốc độ xử lý nhanh hơn Lệnh nhảy: bội số của 4 byte	“Các lệnh có chiều dài thay đổi” 1-16 byte Sử dụng bộ nhớ cache hiệu quả hơn Lệnh có thể có hằng số 8 bit hoặc 32 bit

Mạch số

Điểm khác nhau giữa mạch tổ hợp và mạch tuần tự

- Mạch tổ hợp: giá trị đầu ra chỉ phụ thuộc vào giá trị đầu vào
- Mạch tuần tự: giá trị đầu ra phụ thuộc vào giá trị đầu vào và tín hiệu đầu ra tại thời điểm trước đó (có 1 vài đầu ra nối ngược lại đầu vào)

Các tên cổng và ý nghĩa

Tên cổng	Hình vẽ	Ký hiệu
AND		$x.y$
OR		$x+y$
XOR		$x \oplus y$
NOT		$\bar{x}$ (hoặc x')
NAND		$\overline{x.y}$
NOR		$\overline{x+y}$
NXOR		$\overline{x \oplus y}$

Các thuật toán bool

Các luật cơ bản		
$A \cdot B = B \cdot A$	$A + B = B + A$	Luật giao hoán
$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$	$A + (B \cdot C) = (A + B) \cdot (A + C)$	Luật phân phối
$1 \cdot A = A$	$0 + A = A$	Luật đồng nhất
$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$	Phần tử nghịch đảo
Other laws		
$0 \cdot A = 0$	$1 + A = 1$	
$A \cdot A = A$	$A + A = A$	
$\overline{A \cdot (B \cdot C)} = \bar{A} + \bar{B} + \bar{C}$	$\overline{A + (B + C)} = \bar{A} \cdot \bar{B} \cdot \bar{C}$	Luật kết hợp
$\overline{A \cdot B} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A} \cdot \bar{B}$	Luật DeMorgan



Khoa
CÔNG NGHỆ THÔNG TIN
ĐH Khoa học Tự nhiên TP HCM

Ví dụ rút gọn bằng biểu đồ Karnaugh



x	y	z	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

x\yz	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$f = x.z + x.y + y.z = x.(y + z) + y.z$$

• SOP (Sum Of Products)

– Với bảng chân trị cho mạch n đầu vào $x_1, \dots, x_n$ và một đầu ra f , ta dễ dàng thiết lập công thức (hàm) logic theo thuật toán sau:

- Ứng với mỗi hàng của bảng chân trị có đầu ra bằng 1, viết một tích dạng $u_1.u_2...u_n$ trong đó

$$u_i = \begin{cases} x_i & \text{nếu } x_i = 1 \\ \bar{x}_i & \text{nếu } x_i = 0 \end{cases}$$

- Cộng các tích lại thành tổng, đó chính là công thức của f

• POS (Product Of Sums)

– Trường hợp số hàng có giá trị đầu ra bằng 1 nhiều hơn bằng 0, có thể đặt biến $g = \text{NOT}(f)$, sau đó viết công thức dạng SOP cho g , rồi lấy NOT để có công thức dạng POS (tích của tổng) của f

– Ví dụ

x	y	z	f	g
0	0	0	1	0
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

$$g = \bar{x}.y.\bar{z} + x.\bar{y}.\bar{z}$$

$$f = \bar{g} = (x + \bar{y} + z)(\bar{x} + y + z)$$

12

Mạch cộng:

Lập bảng chân trị, cộng tất cả vào, kết quả ở Sum và dư ở Carry.

• Mạch nửa cộng (Half - adder)

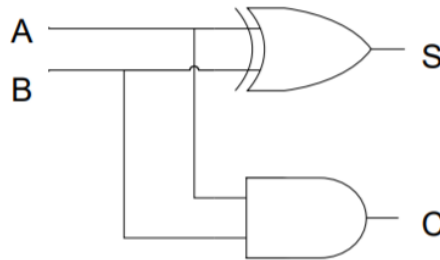
– Bảng chân trị

Cộng 1 bit			
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

– Hàm luận lý

- $\text{Sum} = \bar{A}B + A\bar{B}$
- $\text{Carry} = AB$

– Sơ đồ mạch



Mạch trừ

$$S = A + B + 0$$

$$S = A - B = A + (-B) = A + \bar{B} + 1$$

- Tùy thuộc vào cin đầu tiên mà mạch sẽ thực hiện phép toán gì: nếu cin = 0 thì thực hiện phép cộng còn cin = 1 thì thực hiện phép trừ not b

Mạch mux

Cách chọn mux:

tổng số đầu - số lượng mul + 1 = số đầu gốc

vd 8-1 xài 2 cái 4-1, 1 cái 2-1

$$2 * 4 + 1 * 2 - 3 + 1 = 8$$

- Để chọn n tín hiệu đầu vào thì cần $\log_2(n)$ tín hiệu chọn hay nói cách khác với m tín hiệu chọn có thể chọn tối đa 2^m tín hiệu vào

- Demux dùng để chọn tín hiệu đầu vào sẽ ra ở ngõ nào

Mạch ALU

$$S = 00 \quad R = A + B$$

$$S = 01 \quad R = A - B$$

$$S = 10 \quad R = A \& B$$

$$S = 01 \quad R = A | B$$

- Tách cái 4-1 ra thành 2 cái 2-1 trong ALU để vừa quyết định chọn and/or, vừa quyết định chọn phép toán số học/logic

Sơ lược về các trường trong MIPS



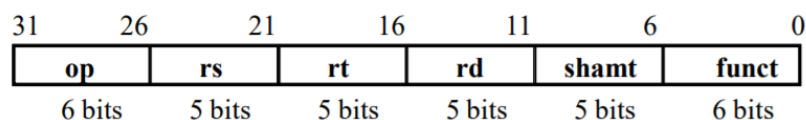
Khoa
CÔNG NGHỆ THÔNG TIN
ĐH Khoa học Tự nhiên TP HCM

Cấu trúc R-Format (2/2)

- rs (Source Register): chứa địa chỉ thanh ghi nguồn thứ 1
- rt (Target Register): chứa địa chỉ thanh ghi nguồn thứ 2 (sai tên ?)
- rd (Destination Register): chứa địa chỉ thanh ghi đích
- Mỗi trường có kích thước 5 bit, nghĩa là biểu diễn được các số từ 0-31 (đủ để biểu diễn 32 thanh ghi của MIPS)

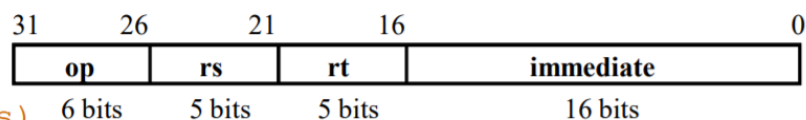
- add, sub, and, or, slt

- add rd,rs,rt
- sub rd,rs,rt
- and rd,rs,rt
- or rd,rs,rt
- slt rd,rs,rt



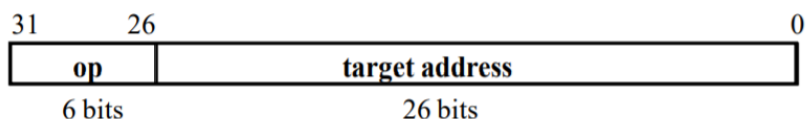
- lw, sw, beq

- lw rt,imm16(rs)
- sw rt,imm16(rs)
- beq rs,rt,imm16



- j

- j addr26



- Các trường

- op (“opcode”): mã thao tác của lệnh, xác định lệnh làm gì
- funct: kết hợp với op (nếu cần) để xác định lệnh làm gì
- rs, rt, rd: địa chỉ các thanh ghi nguồn và đích
- shamt: số bit dịch
- address / immediate: địa chỉ hoặc hằng số tính toán
- target address: địa chỉ cần nhảy tới

- Công đoạn 1: Nạp lệnh (Instruction Fetch)

- Nạp lệnh 32 bit từ bộ nhớ tại địa chỉ trong thanh ghi PC vào thanh ghi lệnh. Công đoạn này như nhau cho tất cả các lệnh
- Sau đó, tăng PC để chuẩn bị nạp lệnh kế tiếp sau khi xử lý xong lệnh này ($PC = PC + 4$)

- Công đoạn 2: Giải mã lệnh (Instruction Decode)

- Phân tích các trường trong lệnh

- Xác định opcode để biết loại lệnh và vị trí của các trường khác
- Sau đó, đọc các thanh ghi nguồn để chuẩn bị thực hiện công đoạn tiếp theo

Ví dụ

- Lệnh add, đọc 2 thanh ghi nguồn
- Lệnh lw, đọc 1 thanh ghi nguồn

• Công đoạn 3: Tính toán (ALU – Arithmetic-Logic Unit)

- Công việc chính của hầu hết các lệnh thực hiện tại công đoạn này: tính toán số học (+, -), luận lý (&, |), so sánh (`beq`, `slt`)
- Lệnh `beq` tính vị trí cần nhảy tới
- Còn lệnh `lw` và `sw` làm gì trong công đoạn này ?
 - `lw $t0, 40($t1)`
 - Địa chỉ của vùng nhớ cần truy xuất = giá trị của `$t1` CỘNG 40
 - Do đó, thực hiện phép cộng trong công đoạn này

• Công đoạn 4: Truy xuất bộ nhớ (Memory Access)

- Thực sự chỉ có lệnh `lw` và `sw` thực hiện công đoạn lệnh này
- Do công việc truy xuất bộ nhớ mất thời gian tương đối nhiều nên cần một công đoạn riêng

• Công đoạn 5: Ghi kết quả vào thanh ghi (Register Write)

- Hầu hết các lệnh đều ghi kết quả tính toán vào một thanh ghi như tính toán số học, luận lý, `lw`, `slt`
- Còn các lệnh `sw`, lệnh nhảy ?
 - Không ghi kết quả gì vào thanh ghi
 - Do đó, các lệnh này không làm gì tại công đoạn lệnh này

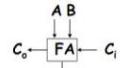
3 công đoạn đầu là bắt buộc:

- + Nạp lệnh
- + Giải mã lệnh
- + Tính toán - xử lý lệnh

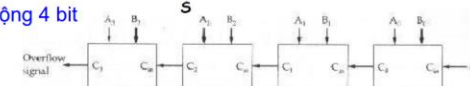
Ví dụ lệnh `add (1,2,3,5)`, lệnh `lw (1,2,3,4,5)`

Mạch cộng (3/3)

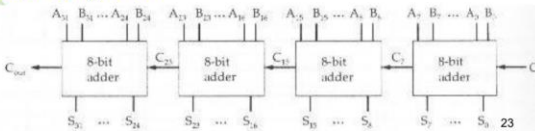
- Mạch cộng 1 bit



- Mạch cộng 4 bit



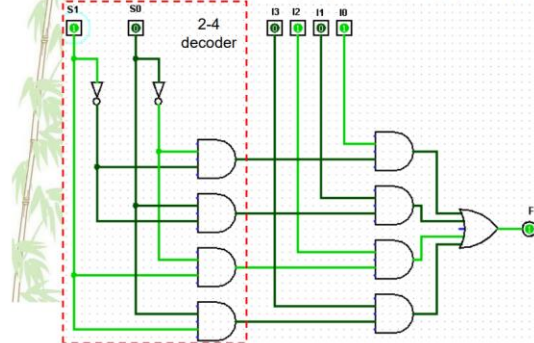
- Mạch cộng 32 bit



MUX – Multiplexor (3/4)

- 4-1 multiplexor

$$F = \overline{S_1} \overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$



32

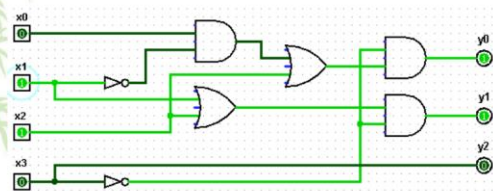
Mạch mã hóa (3/3)

- Mạch mã hóa ưu tiên 4-3 (4-3 Priority Encoder)

$$y_0 = (x_2 + x_0.x_1).x_3$$

$$y_1 = (x_2 + x_1).x_3$$

$$y_2 = x_3$$



Mạch giải mã (2/3)

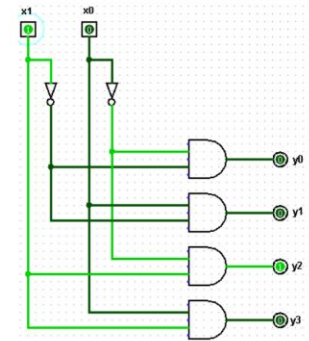
- Mạch giải mã 2-4 (2-4 decoder)

$$y_0 = \overline{x_1}.\overline{x_0}$$

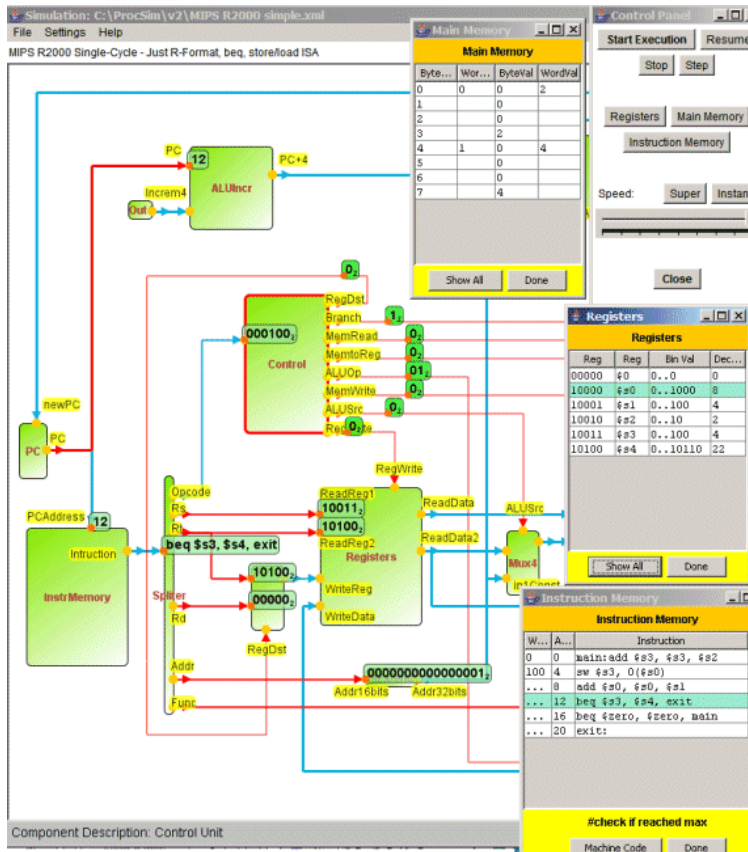
$$y_1 = \overline{x_1}.x_0$$

$$y_2 = x_1.\overline{x_0}$$

$$y_3 = x_1.x_0$$



28



- Thanh ghi PC đưa địa chỉ lệnh theo 2 hướng:
- + Vào instrMemory để lấy ra lệnh đó là lệnh gì là công đoạn nạp lệnh
- + Đưa vào ALUIncr để tính toán địa chỉ lệnh tiếp theo (bth là $PC = PC + 4$)
- Sau khi nạp lệnh sẽ đưa lệnh vào thanh ghi IR(chứa lệnh) rồi đến công đoạn giải mã lệnh. Để giải mã lệnh thì lệnh này được phân tích ra thành các trường theo đúng cấu trúc lệnh của lệnh đó:
- + 6 bit opcode sẽ được đưa vào CU, 5 bit Rs và 5 bit Rt sẽ được đưa vào thanh ghi, 5 bit Rt và 5 bit Rd sẽ được đưa vào Mux, 16 bit immediate, 6 bit function được đưa vào ALUControl
- Vì nó chưa biết đây là lệnh theo cấu trúc gì, nó chỉ rẽ lệnh theo các trường của tất cả các cấu trúc quan trọng
- Dựa vào opcode Cu sẽ được con đường đi cho tất cả lệnh này, nếu giá trị ấy không có trong cấu trúc lệnh này Cu sẽ điều khiển sao cho con đường chứa giá trị đó trở nên vô nghĩa
- Khi các trường chứa địa chỉ lệnh đi vào thanh ghi nó sẽ lấy giá trị của trường đó

Hệ thống lưu trữ (Cache)

CPU gồm 3 thành phần chính: William Stallings

- + Thanh ghi (Registers)
- + ALU
- + CU (Control unit)

Patterson & Hennessy theo góc độ thiết kế CPU

- + CU vẫn như cũ
- + 2 còn lại đều là Datapath (Thanh ghi và ALU)

Bài tập

Assuming a cache of 4K blocks, a four-word block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are direct mapped, two-way and four-way set associative, and fully associative.

32 bit BNC

Fully associative: 32 bit = 30 bit block/tag + 2 bit word

$4K = 4 * 2^{10} = 2^{12}$ line $\rightarrow$ số bit line = 12

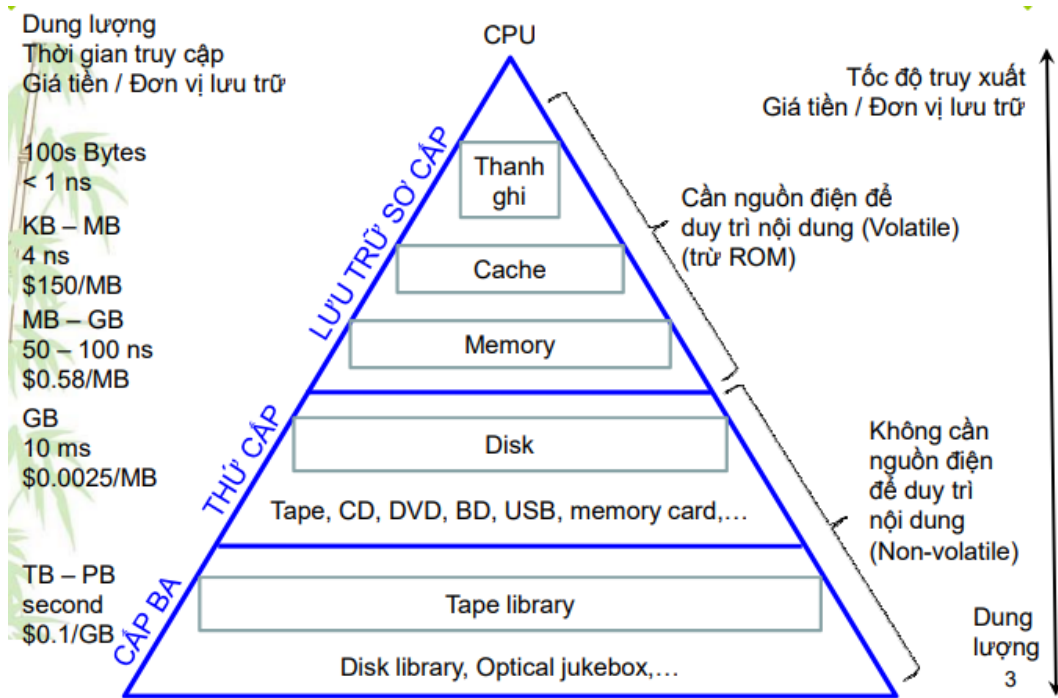
4 word $\rightarrow \log_2(4) = 2$ bit word

Direct mapping: 32 bit = 18 bit tag + 12 bit line + 2 bit word
 $(2^{30} \text{ blocks} / 2^{12} \text{ lines} = 2^{18})$

Two-way set associative: 32 bit = 19 bit tag + 11 bit set + 2 bit word
 $(2^{12} \text{ lines} \rightarrow 2^{12} / 2^1 = 2^{11} \text{ sets} (2 \text{ lines / set}) \quad 2^{30} \text{ blocks} / 2^{11} \text{ sets} = 2^{19})$

four-way set associative: 32 bit = 20 bit tag + 10 bit set + 2 bit word
 $(2^{12} \text{ lines} \rightarrow 2^{12} / 2^2 = 2^{10} \text{ sets} (4 \text{ lines / set}) \quad 2^{30} \text{ blocks} / 2^{10} \text{ sets} = 2^{20})$

****Cách tính số bit set:** số bit line - số bit để biểu diễn set ($\log_2(\text{số set})$) $\rightarrow$ số bit tag = số bit còn dư sau khi biểu diễn hết word, line (set)



1. Lưu trữ sơ cấp

a. Thanh ghi

- Là thiết bị lưu trữ có dung lượng nhỏ nhất nhưng tốc độ truy xuất nhanh nhất
- Được dùng trong các bộ xử lý: Lưu giữ lệnh và dữ liệu (toán hạng, kết quả tính toán, các bit trạng thái) để xử lý
- Có thể được làm bằng nhiều công nghệ khác nhau: trigger, core, thin film,...
- Thường được tổ chức thành tập thanh ghi (register file)

b. Read only memory

- ROM – Là loại bộ nhớ chỉ đọc, không thể ghi, không cần nguồn điện để duy trì nội dung
- PROM – Programmable ROM, ghi được 1 lần duy nhất
- EPROM – Erasable PROM, có thể xóa bằng tia cực tím sau khi ghi
- EEPROM – Electrically EPROM, có thể xóa bằng điện, ghi, xóa từng byte
- FlashROM – Không thể xóa từng byte mà phải xóa từng khối, nhưng tốc độ ghi, xóa rất cao

c. Random access memory

- SRAM - Static RAM (RAM tĩnh)
- DRAM - Dynamic RAM (RAM động)

d. Bộ nhớ chính

- SDRAM (Synchronous RAM): Việc truy xuất được đồng bộ bởi tín hiệu đồng hồ bên ngoài
- Bộ nhớ chính (main memory / “RAM”) được làm từ công nghệ SDRAM
- SDR-SDRAM (Single Data Rate SDRAM)
- DIMM 168-pin (Dual In-line Memory Module)
- Data bus: 64 bit

Standard name	Memory clock	Cycle time	I/O bus clock	Peak transfer rate
SDR-66	66 MHz	15 ns	66 MHz	528 MB/s ($8 \cdot 66 \cdot 10^6 / 10^6$)

SDR-100	100 MHz	10 ns	100 MHz	800 MB/s
SDR-133	133 MHz	7.5 ns	133 MHz	1064 MB/s

- **DDR-SDRAM (Double Data Rate)**

- 1 chu kì truyền được 2 lần data thay vì 1 của SDR-RAM

- e. **Bộ nhớ cache (trọng tâm)**

- Dùng công nghệ SRAM, có tốc độ cao hơn bộ nhớ chính (dùng công nghệ DRAM)
- Đóng vai trò làm bộ nhớ đệm truy xuất nhanh (trung gian giữa CPU và bộ nhớ chính)
- Lưu trữ tạm bản sao một phần nội dung của bộ nhớ chính nhằm giảm truy xuất vào bộ nhớ chính

- **Nguyên lý hoạt động của cache**

Khi CPU / IO cần đọc 1 ô nhớ từ bộ nhớ chính:

- Kiểm tra xem có trong cache chưa?
- Nếu đã có (cache hit): đọc nội dung trong cache, không cần truy xuất bộ nhớ chính
- Nếu chưa có (cache miss): chép khối nhớ chứa ô nhớ cần đọc từ bộ nhớ chính vào cache rồi vào CPU / IO. Thời gian để xử lý cache miss gọi là miss penalty

Theo 2 nguyên lý:

- Cục bộ về thời gian (Temporal locality): Nếu một ô nhớ được dùng đến trong thời điểm hiện tại thì nó dễ có khả năng được dùng đến lần nữa trong tương lai gần
- Cục bộ về không gian (Spatial locality): Nếu một ô nhớ được dùng đến trong thời điểm hiện tại thì những ô lân cận có khả năng sắp được dùng đến

- **Thiết kế bộ nhớ cache**

- Tổ chức bộ nhớ cache
- Chiến lược thay thế
- Đồng bộ hóa nội dung nằm trong cache và bộ nhớ chính
- Kích thước bộ nhớ cache
- Kích thước một phần tử của bộ nhớ cache
- Số lượng và loại cache
- 1 phần tử trong cache được gọi là Line thường lưu 1 block dữ liệu trong bộ nhớ chính, bộ nhớ chính được tổ chức thành các block, khi có sự truy xuất đến 1 byte nào đó trong block thì cả block đó sẽ được cache lại
- Mỗi line sẽ có tag lưu lại thông tin để biết line đó đang chứa dữ liệu block nào
- Truy xuất CPU với Cache theo từng byte
- Truy xuất Cache với RAM theo từng block dữ liệu

- **Tổ chức bộ nhớ cache**

- Fully Associate
- Một block có thể được nạp vào bất kỳ line nào
- Tín hiệu địa chỉ truy xuất bộ nhớ sẽ có cấu trúc như sau

Block, Tag s	Word w
---------------------	---------------

- Tính số bit word rồi tính số bit tag
- Số bit làm Tag phải lưu trong cache nhiều hơn
- Chi phí tìm kiếm cao

- Direct-mapping
- Tín hiệu địa chỉ truy xuất bộ nhớ sẽ có cấu trúc như sau

Tag $s-r$	Cache line r	Word w
-----------------------------	----------------------------------	----------------------------

- Tính số bit word và số bit line, số bit tag = số bit address - bit word - bit line
- Đơn giản
- Chi phí tổ chức thấp
- Không uyển chuyển
- Set Associate
- Kết hợp ý tưởng giữa direct mapping và associate mapping
- Tín hiệu địa chỉ truy xuất bộ nhớ sẽ có cấu trúc như sau

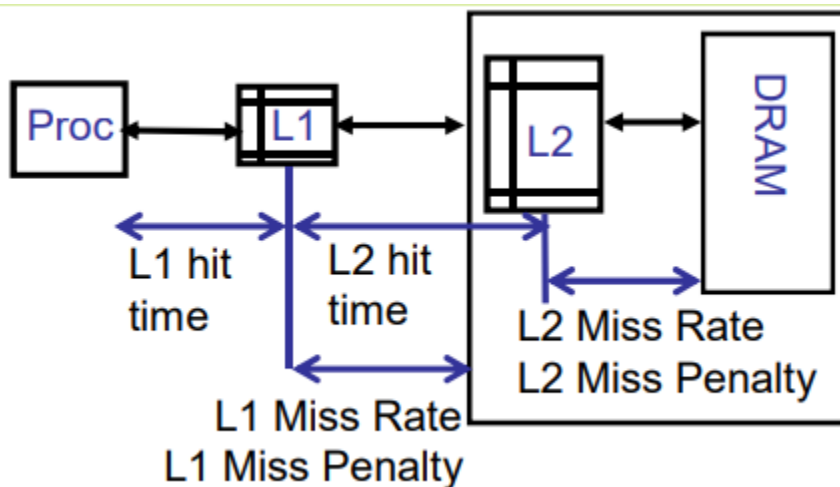
Tag $s-d$	Cache set d	Word w
-----------------------------	---------------------------------	----------------------------

- Tính số bit word
- Bit set = bit line / line per set
- Bit tag = bit address - bit set - bit word
- Số set = số line => direct mapping
- Số set = 1 => fully associative mapping
- **Chiến lược thay thế**
- Khi cần chuyển một block mới vào trong cache mà không tìm được line trống theo yêu cầu của kiểu tổ chức cache thì phải thay line nào ra ?
- Direct Mapping: không có lựa chọn
- Fully Associate Mapping và Set Associate Mapping
 - + Random
 - + FIFO (First In First Out) thằng nào nạp đầu tiên thì được xóa đầu tiên (2 thằng đầu không dựa vào sự truy xuất)
 - + LRU (Least Recently Used) (thay thế phần tử có thời gian lâu nhất kể từ lần cuối sử dụng)
 - + LFU (Least Frequently Used) (thay thế phần tử có tần số truy xuất ít nhất)
- **Đồng bộ hóa cache và bộ nhớ chính**

2 phương pháp

- Write through: mọi thay đổi đều được cập nhật trên bộ nhớ chính và bộ nhớ cache (2 chiều), quá nhiều việc thay đổi dữ liệu nên sẽ chậm
- Write back: chỉ có thay đổi nào cần cập nhật thì mới cập nhật
 - + - Dữ liệu trên cache bị thay đổi thì nó sẽ thay đổi và có thông báo rằng cache ấy đã bị thay đổi
 - + Nếu dữ liệu trên line ấy bị thay đổi, ghi vào bộ nhớ chính nếu nó đã được thông báo bị thay đổi
 - + Mọi truy xuất của CPU đều thông qua bộ nhớ cache
 - + Đây là cách hiện thời mà mọi bộ nhớ cache đều đang sử dụng
- **Kích thước block và cache**
- Kích thước block:
 - + Nhỏ quá: giảm tính lân cận về không gian (spatial locality)
 - + Lớn quá: thời gian chuyển block vào cache lâu (miss penalty)
 - + Kích thước block thường từ 8 tới 64 bytes

- Kích thước cache:
 - + Kích thước cache tỉ lệ thuận với giá thành, có tỉ lệ thuận với tốc độ ?
 - + Nhỏ quá: số lượng block có thể lưu trong cache ít, dẫn đến tỉ lệ cache miss sẽ cao
 - + Lớn quá: nhiều nội dung không cần thiết lưu trong cache. Mất nhiều thời gian để kiểm tra block có nằm trong cache chưa
- **Số lượng và loại cache**
- Thường sử dụng nhiều mức cache L1, L2, L3
 - + On-chip cache: nằm trên cùng IC với bộ xử lý, truy xuất nhanh
 - + Off-chip cache: nằm trên IC riêng và được nối với bộ xử lý qua hệ thống bus (BSB – Back Side Bus), truy xuất chậm hơn
 - + L1 cache: size 10sKB, hit time 1s cycles, miss rate 1-5%
 - + L2: size 100sKB, hit time 10s cycles, miss rate 10-20%
- Thường sử dụng 2 loại cache:
 - + Unified cache: một cache cho cả lệnh và dữ liệu
 - + Split cache: cache riêng cho lệnh (Instruction cache) và dữ liệu (data cache)
- **Mô hình các mức cache**



- $AMAT \text{ (Average memory access time)} = L1 \text{ hit time} + L1 \text{ miss rate} * L1 \text{ miss penalty}$
- $L1 \text{ miss penalty} = L2 \text{ hit time} + L2 \text{ miss rate} * L2 \text{ miss penalty}$. Cứ lần lượt các level sau tương tự
- Càng nhiều level tốc độ truy xuất càng nhanh

- **Writing cache-friendly code**

```

a[M][N]
for (i = 0; i < M; i++)
    for (j = 0; j < N; j++)
=> Miss rate = 25%

for (j = 0; j < N; j++)
    for (i = 0; i < M; i++)
=> Miss rate = 100%
```

2. Lưu trữ thứ cấp

a. Đĩa từ

- Là thiết bị lưu trữ dữ liệu lâu dài phổ biến nhất, gồm một hay nhiều lớp đĩa phẳng được phủ từ để lưu dữ liệu
- 2 loại:
 - + Đĩa mềm (Floppy disks) – chậm, chỉ 1 lớp đĩa
 - + Đĩa cứng (Hard Disk Drives - HDD) – nhanh hơn, nhiều lớp đĩa
- b. Tổ chức đĩa cứng
 - Gồm nhiều lớp đĩa phủ từ 1 hoặc cả 2 mặt, mỗi mặt có 1 đầu đọc dữ liệu, mỗi mặt có rất nhiều tracks, mỗi tracks có nhiều số hiệu gọi là cylinders, mỗi track chia thành sectors, mỗi sector 4096 bit (0.5 KB)

Tính dung lượng

1 platter, 2 heads, 80 tracks/head, 18 sectors/track

Dung lượng = 2 heads/disk * 80 tracks/head * 18 sectors/track

= 2880 sector/disk

= 0.5KB/sector * 2880 sector/disk

= 1440 KB/disk (~ 1.44MB)

- c. Cơ chế đọc dữ liệu của đĩa cứng
 - Disk latency = seek time + rotation time + transfer time
- d. Kỹ thuật RAID (Redundant Array of Inexpensive Disks)
 - Kết hợp nhiều ổ đĩa (vật lý) thành một hệ thống đĩa (luận lý) duy nhất bằng phần cứng (RAID controller) hoặc phần mềm
 - Mục đích:
 - + Đảm bảo an toàn dữ liệu
 - + Tăng tốc độ truy xuất của hệ thống
 - Hoạt động:
 - + Dữ liệu lưu trên đĩa sẽ được lưu đồng thời lên tất cả các đĩa
 - + Hoạt động của hệ thống trong suốt với người dùng
- Cấu hình 1 số loại RAID
 - RAID 5 lưu 1 bản A_p
 - RAID 6 lưu 2 bản A_p và A_q
 - Cơ chế kiểm tra lỗi và sửa lỗi
 - Nguyên nhân gây ra lỗi: do các xung điện hoặc điện từ trường..
 - Phương pháp:

7 bits of data (number of 1s)	8 bits including parity	
	even	odd
0000000 (0)	00000000	10000000
1010001 (3)	11010001	01010001
1101001 (4)	01101001	11101001
1111111 (7)	11111111	01111111

- + Parity bit:

Hệ thống nhập xuất

Một số thiết bị nhập xuất

– Chuột, màn hình, bàn phím, máy in, đĩa cứng, loa, modem, các mạng, ...

Vai trò của thiết bị nhập xuất

– Giao tiếp giữa thành phần xử lý của máy tính với các đối tượng bên ngoài giúp các đối tượng này có thể cung cấp yêu cầu và dữ liệu cho thành phần xử lý.

Một số tiêu chí phân loại thiết bị nhập xuất

- Chức năng: Nhập – Xuất – Lưu trữ
- Đối tượng tương tác: Người dùng – Thiết bị khác
- Tốc độ nhập xuất

Tổ chức kết nối các thiết bị nhập xuất

– Nhiều thiết bị nhập xuất với tốc độ nhập xuất chênh lệch nhau, chênh lệch với tốc độ truy xuất của CPU.
VD: Bộ xử lý 1GHz có thể xử lý 1 tỉ lệnh truy xuất bộ nhớ trong 1 giây, tương đương tốc độ truy xuất dữ liệu 4,000,000 KB/s

– Do đó, hệ thống máy tính thường sử dụng nhiều cấp độ bus khác nhau để kết nối các nhóm thiết bị nhập xuất. VD: PCI, PCIe, AGP, USB, IDE, SATA

– Bus thường gồm 3 phần: bus địa chỉ, bus dữ liệu, bus điều khiển

Truy xuất thiết bị nhập xuất

Bộ xử lý truy xuất các thiết bị nhập xuất bằng cách nào?

Port-mapped: Mỗi thiết bị được gán một hoặc vài port. Truy xuất vào các port này sẽ tương ứng với truy xuất thiết bị. Trong bộ lệnh của bộ xử lý thường phải có một số lệnh chuyên biệt để thao tác với thiết bị nhập xuất • Ví dụ: lệnh IN và OUT trên kiến trúc x86

Memory-mapped: Mỗi thiết bị được ánh xạ vào một hoặc vài vùng nhớ. Truy xuất vào những vùng nhớ này sẽ tương ứng với truy xuất thiết bị • Ví dụ: dùng lệnh lw và sw để truy xuất các thiết bị trên kiến trúc MIPS

3 cơ chế giao tiếp giữa CPU và thiết bị nhập xuất

– Polling (còn gọi là Programmed I/O) – Interrupt-driven – Direct memory access (DMA)

Polling

CPU điều khiển toàn bộ quá trình nhập xuất.

- Kiểm tra tình trạng thiết bị
- Gửi yêu cầu truy xuất
- Truyền dữ liệu

CPU phải chờ trong suốt thời gian truy xuất thiết bị

Interrupt-driven

CPU không phải chờ trong suốt quá trình truy xuất thiết bị

Chi phí xử lý ngắt

DMA

Khi truyền dữ liệu lớn, phải ngắt CPU nhiều lần (trên mỗi đơn vị truyền dữ liệu)

Direct memory access

- Hệ thống hỗ trợ thành phần DMA Controller để giao tiếp với các thiết bị cần truy xuất khối lượng lớn dữ liệu
- Chỉ ngắt CPU một lần cho một yêu cầu truy xuất dữ liệu

Bảng lũy thừa của 2

n	2^n	2^{-n}
0	1	1
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
5	32	0.03125
6	64	0.015625
7	128	0.0078125
8	256	0.00390625
9	512	0.001953125
10	1024	0.0009765625
11	2048	0.00048828125